CS344 (OS Lab) Assignment 3

Group 6 Members:

Adarsh Gupta	220101003
Aditya Nanda	220101006
Aryan Raj	220101019
Ashish Raj	220101020

Table of Contents:

Task A: Lazy Memory Allocation 2
Task B: Adding LRU Swapping in xv6 4
Question-Answers 4
Task 1: Adding Kernel Process Function 5
Task 2: Swapping Out Pages 6
Task 2.1: Creating circular queues 6
Task 2.2: Instantiating a waiting queue 7
Task 2.3: Making the process sleep 8
Task 2.4: File Management Functions 9
Task 2.5: Swapout Pages 9
Task 2.6: Wakeup sleeping processes 12
Task 2.7: Simulating LRU Swapping 12
Task 2.8: Killing the clearSwapOutRequiredQueue process 13
Task 3: Swapping In Pages 14
Task 3.1: Instantiating a SwapInWaitingQueue 14
Task 3.2: Handling Page Fault 14
Task 3.3: Swap In Pages 16
Task 4: Sanity Checking 19

Code Patch Location:

CS344Assignment3Group6
(https://drive.google.com/drive/folders/1q3VRMFIFp7O5azPek_bFuzrIauR5demg)

Task A: Lazy Memory Allocation

Any running process can make the sbrk system call to request more memory from the OS in xv6. sbrk uses growproc() (defined in proc.c). growproc() calls allocuvm() which is responsible for allocating the extra pages and adding the additional mapping information to the page table of the process.
In this task, we have to add support for Lazy Memory Allocation, i.e. only allocate memory when it is accessed and not when it is requested.

To add lazy memory allocation, the following sequence of steps was followed:

1) Removing memory allocation from growproc, and instead giving a false sense of growth to process by just incrementing the size.
   sysproc.c:

```
int
sys_sbrk(void)
{
  int addr;
  int n;

  if(argint(0, &n) < 0)
    return -1;
  addr = myproc()->sz;

  // Giving false sense of memory allocation to process
  myproc()->sz = myproc()->sz + n;

  // if(growproc(n) < 0)
  //   return -1;
  return addr;
}
```

2) Trapping on memory page fault, and allocate a page lazily
   When the process accesses the page that it thinks it has access to it throws a page fault, and traps to the kernel with trap number T_PGFLT (14).

```
switch(tf->trapno){
case T_PGFLT:
  // handling page fault by lazy memory allocation
  if(lazyAllocate() < 0){
    cprintf("Failed to lazily allocate memory.\n");
    panic("memory");
  }
```

   The trap is handled by calling lazyAllocate, which uses the virtual address at which the fault occurred (obtained by the inline assembly function rcr2 which reads the cr2 register) and creates a page in virtual memory with that address. This is done by rounding the page fault address, and then allocating a fresh free page in the memory using kalloc. This free physical page is then cleared and mapped to the required rounded address in the virtual memory of the process, this is done by using mappages.

```
int lazyAllocate(){
  // Get page fault address
  int addr=rcr2();

  // Get page address using PGROUNDDOWN address
  int rounded_addr = PGROUNDDOWN(addr);

  // Try to allocate a page
  char *mem=kalloc();
  if(mem!=0){
    // If memory allocation succedded, set it to 0
    memset(mem, 0, PGSIZE);
    // Add mapping in the processes page table
    if(mappages(myproc()->pgdir, (char*)rounded_addr, PGSIZE, V2P(mem), PTE_W|PTE_U) >= 0)
      // Successful allocation and mapping
      return 0;
  }

  return -1;
}
```

3) Change mappages from static int to int in vm.c, and add a declaration of mappages in trap.c. This is done to allow usage of mappages in trap.c while mapping the newly lazily allocated page at the desired address.

trap.c:

```
int mappages(pde_t *pgdir, void *va, uint size, uint pa, int perm);
```

The parameters of mappages is the page directory of the process (pgdir), virtual address of the start of the data, size of the data, physical memory at which the physical page resided and permission corresponding to the page table entry.

Task B: Adding LRU Swapping in xv6

Question-Answers

Q1) How does the kernel know which physical pages are used and unused?
In kalloc.c which contains functions responsible for all aspects of page manipulation, there is a struct kmem which contains a linked list of all the free pages currently available. This linked list is used by kalloc to allocate pages by acquiring the lock and taking the first entry from the list (if it is not null) and moving the head 1 ahead. The pages that get free (by kfree) are attached at the end of this linked list.

kalloc.c

```
struct {
  struct spinlock lock;
  int use_lock;
  struct run *freelist;
} kmem;
```

kalloc:

```
  r = kmem.freelist;
  if(r)
    kmem.freelist = r->next;
```

kfree:

```
  r = (struct run*)v;
  r->next = kmem.freelist;
  kmem.freelist = r;
```

Q2) What data structures are used to answer this question?
A linked list named kmem can be used to answer at any point in time which physical pages are used and unused.

Q3) Where do these reside?
This linked list structure resides inside kalloc.c and is part of the kmem struct. Each node is a run struct which is also defined in kalloc.c

Q4) Does xv6 memory management limit the number of user processes?
The process table in xv6 is static and is allocated at startup. By default, the process table consists of NPROC (64) entries which can be reused. This limits the number of user processes, however from memory management point of view, the number is much higher, by default the maximum physical memory is 240 MB and assuming each process takes 1 MB (towards the higher end, obviously have to use estimates here because of high variability) there can be atleast 240 processes, again this number is highly variable but is more than the standard 64 processes limit imposed by process table size.

Q5) If so, what is the lowest number of processes xv6 can ‘have’ at the same time (assuming the kernel requires no memory whatsoever)?
After bootup, the init process always starts, therefore there is atleast 1 process. If by some code, we use up the entirety of the physical memory (by default 240 MB) before even forking shell in the init process, we can, technically speaking, have just 1 process which is the minimum number of processes that xv6 can have at a time. From a general pov, xv6 atleast consists of 2 processes, the init process and the shell process.

Task 1: Adding Kernel Process Function

This function creates a new kernel process and makes it execute a void function provided as the argument.

defs.h:

```
void            create_kernel_process(const char*, void (*)());
```

proc.c:

```
536
537 // create_kernel_process(const char*, void (*f)())
538 // This function takes in process name and a void function pointer
539 // It allocated a new kernel process (a process that only runs in kernel mode)
540 // and makes it execute the function pointer provided
541 void
542 create_kernel_process(const char* name, void (*entrypoint)()){
543   // Allocate a process
544   struct proc* p;
545   p = allocproc();
546   if(p == 0){
547     panic("kernel process creation failed");
548   }
549
550   // Set up the kernel table in the page table
551   p->pgdir = setupkvm();
552   if(p->pgdir == 0){
553     panic("setupkvm failed");
554   }
555
556   // No need to initialse trapframe since this is a kernel process
557
558   // Set eip in context to entrypoint
559   p->context->eip = (uint)entrypoint;
560
561   // Set process name
562   safestrcpy(p->name, name, sizeof(name));
563
564   // Set process as Runnable
565   acquire(&ptable.lock);
566   p->state = RUNNABLE;
567   release(&ptable.lock);
568 }
```

The create_kernel_process(const char*, void(*)()) function was added in proc.c. This function takes a void function pointer with no arguments, creates a new kernel process (a process that always stays in kernel mode), and makes the newly created process execute the void function. Since this process always remains in kernel mode there is no need to fill up its trapframe. All we need to do is fill in the eip in the context to the function pointer provided to execute whatever code needs to be executed by the kernel process.
This function does the following:

- Uses allocproc to create the memory image of the process and add in the process table
- Uses setupkvm to setup the kernel part of the processes page table and sets eip to the function pointer given.

Task 2: Swapping Out Pages

Requirement: Any process that fails to get memory needs to be put to sleep, and added to a waiting queue. A swap out function will be responsible for swapping out a page by saving it to the disk and waking up all the processes sleeping for memory allocation. The swap out function will satisfy all the waiting processes for allocating memory and then kill itself.
The implementation of this entire task has been broken into various subtasks.

Task 2.1: Creating circular queues

All processes that fail to allocate memory must be put to sleep on a channel and added to a circular queue. This subtask covers the implementation of a general circular queue in xv6.

defs.h:

```
struct proc*    circularQueuePop(struct circularQueue*);
int             circularQueuePush(struct circularQueue*, struct proc*);
```

proc.c:

```
// Adding circular queue struct of processes reqd. for swapping out
struct circularQueue {
  // Spinlock for synchronization amongst processes
  struct spinlock lock;
  // Circular queue
  struct proc* queue[NPROC + 1];
  int start;
  int end;
};
```

```
// Pop function for circular queue of processes
struct proc*
circularQueuePop(struct circularQueue* queue){
  // Acquire lock over circular queue
  acquire(&queue->lock);

  // Check if queue is empty
  if(queue->start == queue->end){
    release(&queue->lock);
    return 0;
  }

  // Get process in front
  struct proc* p;
  p = queue->queue[queue->start];

  // Move start
  queue->start += 1;
  queue->start %= NPROC;

  // Release lock over circular queue
  release(&queue->lock);

  return p;
}
```

```
// Push function for circular queue of processes
int
circularQueuePush(struct circularQueue* queue, struct proc* p){
  // Acquire lock over circular queue
  acquire(&queue->lock);

  // Check if queue is full
  if((queue->end + 1)%NPROC == queue->start){
    release(&queue->lock);
    return 0;
  }

  // Add to queue
  queue->queue[queue->end] = p;

  // Move end
  queue->end += 1;
  queue->end %= NPROC;

  // Release lock over circular queue
  release(&queue->lock);

  return 1;
}
```

I have defined a general abstraction of the process circular queue and the corresponding push and pop functions for the struct. I have also added the declaration in defs.h.

Task 2.2: Instantiating a waiting queue

We need to create an instance of a circular queue to which the sleeping processes will be added which failed to allocate memory for themselves.

defs.h:

```
extern struct circularQueue swapOutRequiredQueue;
```

proc.c: Create a struct variable for the processes waiting for swap out

```
// Swap Out Required Queue
// All processes that failed to allocate memory for themselves are sleeping and present on this queue
// waiting for someone to swapout pages and wake them up
struct circularQueue swapOutRequiredQueue;
```

proc.c/pinit: The locks need to be instantiated here

```
void
pinit(void)
{
  initlock(&ptable.lock, "ptable");
  initlock(&swapOutRequiredQueue.lock, "swap out queue");
}
```

proc.c/userinit: The start and end values are instantiated here

```
// Instantiate start and end for swapOutRequiredQueue
acquire(&swapOutRequiredQueue.lock);
swapOutRequiredQueue.start = 0;
swapOutRequiredQueue.end = 0;
release(&swapOutRequiredQueue.lock);
```

Task 2.3: Making the process sleep

Any process which fails to allocate memory needs to sleep on some channel and get added to the swapOutRequiredQueue by someone.
We will start by creating a channel for the process to sleep on.

vm.c:

```
#include "spinlock.h"

struct spinlock swapOutChanelLock;
// To maintaing count of number of sleeping processes
int numSleepingProcesses = 0;
char* swapOutChannel;
```

defs.h:

```
extern struct spinlock swapOutChanelLock;
extern int      numSleepingProcesses;
extern char*    swapOutChannel;
```

proc.c/pinit:

```
  initlock(&swapOutChanelLock, "sleeping channel lock");
```

Anytime a process allocates memory, it does so by using allocuvm function in vm.c which further calls kalloc to allocate a page, kalloc returns 0 if the memory is failed to be allocated. We will add our code to allocuvm to make the process sleep on swapOutChannel.

vm.c/allocuvm:

```
  if(mem == 0){
    deallocuvm(pgdir, newsz, oldsz);

    // If failed to allocate memory, we need to

    // 1) put this process to sleep on swapOutChannel &
    myproc()->state = SLEEPING;
    acquire(&swapOutChanelLock);
    myproc()->chan = swapOutChannel;
    numSleepingProcesses += 1;
    release(&swapOutChanelLock);

    // 2) add it to swapOutRequiredQueue
    circularQueuePush(&swapOutRequiredQueue, myproc());

    return 0;
  }
```

Task 2.4: File Management Functions

The file manipulation functions cannot be called in proc.c, however for the sake of this assignment we still need to use them. The following functions were copied directly with some argument changes from sysfile.c and prepended with proc to signify that they are being used by a kernel process in proc.c.

```
#include "fcntl.h"
#include "fs.h"
#include "file.h"
#include "stat.h"

// For kernel file management
int proc_close(int fd);
int proc_write(int fd, char *text, int size);
static struct inode* proc_create(char *path, short type, short major, short minor);
static int proc_fdalloc(struct file *f);
int proc_open(char *path, int omode);
int proc_read(int fd, int n, char *p);
```

Task 2.5: Swapout Pages

In this part, we will create a function clearSwapOutRequiredQueue() which can run as a kernel process and clear the swapOutRequiredQueue by swapping out old pages by saving them as files. We will also start this process (if not already running) anytime a new process gets added to swapOutRequiredQueue due to memory allocation failure.

defs.h:

```
void            clearSwapOutRequiredQueue();
extern int      swapOutProcessRunning;
```

proc.c:

```
int swapOutProcessRunning = 0;
```

The swapping out kernel process runs a loop until the swap out required queue is empty. When the queue is empty, it executes a set of instructions to terminate itself. In each iteration of the loop a process is extracted from the queue of waiting processes, and we use LRU to determine which page to remove from this process's page table.

For each process, the outer iteration is through each entry of the page directory of the process (i.e. the outer page table), we skip this page table if it has its access bit set, denoting it was accessed recently (The accessed bit are unset every time the process is scheduled, this is covered in Task 2.7: Simulating LRU Swapping). With a high probability, there will exist some page table entries with its access bit unset.

For all such secondary page table which have their access bit unset we iterate through the table and find 1 entry such that its present AND it has not been accessed. When we find such a page entry in the secondary page table, this is the page that needs to be swapped out and freed.

Using the pid and outer page table entry number and inner page table entry number, we calculate the virtual address of the page. These two things are then used along with some string manipulation in the getFileName function to get the filename of the swap file.

Using the file manipulation functions created above, we use the proc_open call with O_CREATE (i.e. create the file if not present) and O_RDWR (i.e. read write permissions) mode. This opens the file and stores the file descriptor in fd.

Using this file descriptor and proc_write system call we write all the data in the page.

This empty page is then marked as free (done by calling kfree) so it is made available to use to all the waiting processes.
As mentioned in the assignment, we need a way to tell if a page has been swapped out earlier. This is done by setting the 8th bit from the right in the secondary page table entry for this page.
The main loop breaks when all the memory needs of all the processes have been satisfied. We clear all the values except the kernel stack. The responsibility of cleaning the kernel stack is given to the scheduler and is discussed in Task 2.8.

Given below is the entire code of clearSwapOutRequiredQueue function with comments.

```
// clearSwapOutRequiredQueue()
// This function is started as a kernel process when memory allocation fails for some process.
// This function saves an old page of a process in the disk and calls kfree to free that page
// kfree is further responsible for waking up the sleeping processes
// This function is active as a kernel process until all memory needs of the waiting processes are satisified.
void
clearSwapOutRequiredQueue(){
  // Acquire lock over swapOutRequiredQueue
  acquire(&swapOutRequiredQueue.lock);

  // Iterate through all sleeping processes
  while(swapOutRequiredQueue.start != swapOutRequiredQueue.end){
    // Get process from queue
    struct proc *p;
    p = circularQueuePop(&swapOutRequiredQueue);

    // Iterate through all page directory entries of this process
    pde_t* pdentry = p->pgdir;
    for(int i = 0; i < NPDENTRIES; i++){

      // If page directory accessed then skip
      if(pdentry[i]&PTE_A) // PTE_A defined in mmu.h and used to extract the 6th bit of the page flags, representing accessed
        continue;

      // When you find a not recently accesed page directory, iterate through that to find a physical page
      pte_t* pgentry;
      pgentry = (pte_t*)P2V(PTE_ADDR(pdentry[i]));

      for(int j = 0; j < NPTENTRIES; j++){
        // Skip if accessed or not present
        if(pgentry[j]&PTE_A || !(pgentry[j]&PTE_P))
          continue;

        // Now we have found an old page that we can remove
        pte_t* pageToRemove = (pte_t*)P2V(PTE_ADDR(pgentry[j]));
```

```
        // Now we have found an old page that we can remove
        pte_t* pageToRemove = (pte_t*)P2V(PTE_ADDR(pgentry[j]));

        // Get filename
        char fileName[50];
        getFileName(fileName, p->pid, i, j);

        // Open file
        int fd=proc_open(fileName, O_CREATE | O_RDWR);
        if(fd<0){
          cprintf("error creating or opening file: %s\n", fileName);
          panic("swap_out_process");
        }

        // Write to File
        if(proc_write(fd,(char *)pageToRemove, PGSIZE) != PGSIZE){
          cprintf("error writing to file: %s\n", fileName);
          panic("swap_out_process");
        }
        proc_close(fd);

        // Mark the page as free
        kfree((char*)pageToRemove);
        memset(&pgentry[j],0,sizeof(pgentry[j]));

        // Mark this page as being swapped out by using the earlier unused 7th bit
        pgentry[j]=((pgentry[j])^(0x080));

        // Now that we have used this page table, we will use another next time
        break;
      }
    }
  }

  // Release lock over swapOutRequiredQueue once all done
  release(&swapOutRequiredQueue.lock);
```

```
  // Release lock over swapOutRequiredQueue once all done
  release(&swapOutRequiredQueue.lock);

  // Killing its own kernel process
  struct proc* p;
  p = myproc();
  if(p == 0)
    panic("swapOutProcess died");

  swapOutProcessRunning = 0;
  p->parent = 0;
  safestrcpy(p->name, "oldSwapOut", len("oldSwapOut"));
  p->killed = 0;
  p->state = UNUSED;
  sched();
}
```

vm.c/allocuvm: (if condition when memory allocation failed after putting proc to sleep)

```
// Call clearSwapOutRequiredQueue if not already
if(!swapOutProcessRunning){
  swapOutProcessRunning = 1;
  create_kernel_process("SwapOutProcess", &clearSwapOutRequiredQueue);
}
```

This makes sure that create_kernel_process can only start clearSwapOutRequiredQueue atmost once at a time, i.e. start only if it does not exist. This is simply controlled using a boolean variable swapOutProcessRunning which signifies whether this process is currently running or not.

The getfilename function, takes a character pointer and populates it with the required filename by using the pid of the process and the page directory number and the page entry number.

Task 2.6: Wakeup sleeping processes

Anytime a memory page gets free (as done by swapping out) by calling kfree, we need to wakeup all the sleeping processes.

kalloc.c/kfree:

```
// Wake up all the processes sleeping on swapOutChannel
if(kmem.use_lock)
  acquire(&swapOutChanelLock);
if(numSleepingProcesses > 0){
  wakeup(swapOutChannel);
  numSleepingProcesses = 0;
}
if(kmem.use_lock)
  release(&swapOutChanelLock);
```

Anytime a free page is avalaible, all the processes that happen to be sleeping on swapOutChannel are woken up. Essentially all processes that were earlier put to sleep due to unavalaibilty of pages are woken up by using the wakeup system call.

Task 2.7: Simulating LRU Swapping

We can simulate LRU swapping by clearing the accessed bit every time the process is scheduled. This will ensure only the old pages are swapped out.

proc.c/scheduler:

```
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
  if(p->state != RUNNABLE)
    continue;

  // Clear the accessed bit of the process
  for(int i=0;i<NPDENTRIES;i++){
    //If PDE was accessed earlier
    if(((p->pgdir)[i])&PTE_P && ((p->pgdir)[i])&PTE_A){
      // We need to unset all accessed bits of the subtable
      // Iterating through the page directory entry &
      pte_t* pgentry = (pte_t*)P2V(PTE_ADDR((p->pgdir)[i]));
      for(int j=0;j<NPTENTRIES;j++){
        // unsetting the accessed bits
        if(pgentry[j]&PTE_A){
          pgentry[j]^=PTE_A;
        }
      }
      // Unsetting accessed bit of page directory
      ((p->pgdir)[i])^=PTE_A;
    }
  }
```

Whenever a process is being context-switched into by the scheduler, all its accessed bits are unset. This code essentially iterates through the page directory, if it finds an accessed secondary page table entry, it unsets all entries in the secondary page table by doing xor operation.

Task 2.8: Killing the clearSwapOutRequiredQueue process

This kernel process needs to be killed after it has successfully satisfied all the memory requests by emptying the swapOutRequiredQueue. The function sets its name as “oldSwapOut” and calls the scheduler. We can use this information in the scheduler to kill this process.

proc.c/scheduler:

```
// Loop over process table looking for process to run.
acquire(&ptable.lock);
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
  // Killing the clearSwapOutRequiredQueue
  if(p->state == UNUSED && !strncmp(p->name, "oldSwapOut", strlen("oldSwapOut"))){
    kfree(p->kstack);
    p->kstack = 0;
    p->name[0] = 0;
    p->pid = 0;
  }
```

All requirements given in the assignment for this task have been covered. For the last requirement stating that only user space memory can be swapped out, note that we are iterating in the page directory of the process starting from 0. This ensures that we will swap out the first user space page that was not accesses, since its not possible for the user program to access its entire memory in between timer interrupts.

Task 3: Swapping In Pages

Anytime a process requests a page and it results in a page fault, it could be due to one of two reasons:

1) Either the page does belong to the memory image but was earlier swapped out
2) Or, process is accessing illegal memory

All the processes which face a page fault, need to be saved to a queue and analyzed, if the page fault is due to an earlier swap out, we need to bring back the page.

Task 3.1: Instantiating a SwapInWaitingQueue

All processes that encounter a page fault due to their page being swapped out need to be placed on a queue, their requests will be resolved later by the clearSwapInQueue kernel process.

defs.h:

```
extern struct circularQueue swapInWaitingQueue;
```

proc.c: Initialise a circular queue variable for the process waiting for a swap in

```
// Swap In Waiting Queue
// All processes that faced a page fault due to page being swapped out are sleeping and present on this queue
struct circularQueue swapInWaitingQueue;
```

proc.c/pinit: The locks needs to be initialised

```
initlock(&swapInWaitingQueue.lock, "swap in queue");
```

proc.c/userinit: The start and end needs to be initialised

```
// Instantiate start and end for swapInWaitingQueue
acquire(&swapInWaitingQueue.lock);
swapInWaitingQueue.start = 0;
swapInWaitingQueue.end = 0;
release(&swapInWaitingQueue.lock);
```

Task 3.2: Handling Page Fault

Whenever a page fault occurs, we need to check if the swapped out bit is set in that page. That bit indicates if the page was swapped out in the past helping us distinguish between the 2 types of page fault discussed above.

We need to store the information of where the page fault occurred in the process struct itself, since this information will be used at multiple places. In case of page fault due to swapping out we need to put the process to sleep and place it in the swapInWaitingQueue, and also start (if not already running) the clearSwapInQueue function.

proc.h/struct proc: Adding variable to store address of page fault

```
int pfAddr;                  // Virtual Address of Pagefault
};
```

trap.c/trap: Handling page fault

```
switch(tf->trapno){
case T_PGFLT:
  handlePageFault();
```

proc.c: To ensure only 1 swapIn process runs at a time

```
int swapInProcessRunning = 0;
```

defs.h:

```
void            clearSwapInQueue();
extern int      swapInProcessRunning;
```

```
extern struct spinlock swapInLock;
```

trap.c:

```
// Lock over which the processes sleep
struct spinlock swapInLock;

void
handlePageFault(){
  // Get virtual address of page fault
  int addr = rcr2();

  // Get current process
  struct proc *p;
  p = myproc();

  // Acquire lock and sleep
  acquire(&swapInLock);
  sleep(p, &swapInLock);

  // Get to the appropriate page table entry to check the type of fault
  pde_t *pde = &(p->pgdir)[PDX(addr)];
  pte_t *pgtab = (pte_t*)P2V(PTE_ADDR(*pde));

  if((pgtab[PTX(addr)])&0x080){
    // This means that the page was swapped out.
    // Store the page fault virtual address in the proc struct
    p->pfAddr = addr;

    // Push the process onto swapInWaitingQueue
    circularQueuePush(&swapInWaitingQueue, p);

    // Start clearSwapInQueue if not already running
    if(!swapInProcessRunning){
      swapInProcessRunning = 1;
      create_kernel_process("swap_in_process", &clearSwapInQueue);
    }
  }

  // Else the page fault was due to illegal access, so program must exit
  exit();
}
```

The first step is to find the address of the page fault using rcr2 (read cr2) inline assembly function. Then add the current process to sleep with a new lock. Then obtain the page table entry using the hierarchical page table and macros already defined in xv6.
Now that we have the page table entry corresponding to the virtual address requested, we need to check if it was swapped out by checking the 8th bit by doing logical and. If it is set we add the process to the swapInWaitingQueue and start the kernel process (if not already running) to solve the requests of this queue.

Task 3.3: Swap In Pages

The clearSwapInQueue function is started as a kernel process when a process encounter a page fault due to an earlier swapping out. This process is present in the swapInWaitingQueue along with any other such process. This kernel process goes through all elements in this queue, and handles their respective requests by opening the file from the disc, reading its content in a newly allocated page and then mapping that newly allocated page to this process' page table.

proc.c:

```
// clearSwapInQueue()
// This function is called when a process encounters page fault due to its page
// being swapped out earlier.
// It extracts a process waiting for page read, allocates a page using kalloc
// then fills in the page using the swapped out earlier.
// Then it uses mappages to add the newly allocated page in its page table
// Then it wakes up the process and after all the proceeses are done it stops itself
void
clearSwapInQueue(){
  // Acquire lock over swapInWaitingQueue
  acquire(&swapInWaitingQueue.lock);

  // Iterate through all elements
  while(swapInWaitingQueue.start != swapInWaitingQueue.end){
    // Get proc in front of the queue;
    struct proc *p;
    p = circularQueuePop(&swapInWaitingQueue);

    // Get filename of the saved file
    char fileName[50];
    int_to_string(p->pid, fileName);
    int x = strlen(fileName);
    fileName[strlen(fileName)] = '_';
    int_to_string(PTE_ADDR(p->pfAddr), fileName + x + 1);
    safestrcpy(fileName + strlen(fileName), ".swp", 5);

    // Open the file and raise error if not exists
    int fd=proc_open(fileName, O_RDONLY);
    if(fd<0){
      release(&swapInWaitingQueue.lock);
      cprintf("could not find swapped out page file in memory: %s\n", fileName);
      panic("swap in illegal access");
    }

    // Allocate a new page and fill the page data in it
    char *mem = kalloc();
    proc_read(fd, PGSIZE, mem);

    // Add the new page to the page table of the process
    if(mappages(p->pgdir, (void *)PTE_ADDR(p->pfAddr), PGSIZE, V2P(mem), PTE_W|PTE_U) < 0){
      release(&swapInWaitingQueue.lock);
      panic("failed to mappages while swapping in");
    }
```

```
    // Wakeup this sleeping process
    wakeup(p);
  }

  // After all processes are done, release the lock
  release(&swapInWaitingQueue.lock);

  // Kill its own kernel process
  struct proc *p;
  p = myproc();
  if(p == 0)
    panic("swapInProcess died");

  swapInProcessRunning = 0;
  p->parent = 0;
  safestrcpy(p->name, "oldSwapIn", strlen("oldSwapIn") + 1);
  p->killed = 0;
  p->state = UNUSED;
  sched();
}
```

```
int mappages(pde_t *pgdir, void *va, uint size, uint pa, int perm);
```

proc.c/scheduler:

```
        // Killing the clearSwapInQueue process
        if(p->state == UNUSED && !strncmp(p->name, "oldSwapIn", strlen("oldSwapIn") + 1)){
          kfree(p->kstack);
          p->kstack = 0;
          p->name[0] = 0;
          p->pid = 0;

        }
```

The kernel process runs a loop till the circular queue is empty. In each iteration, it pops an element from the queue. Then using the pid and fault address (stored in proc struct), using string manipulation we get the name of the file that the page data is stored in. Using proc_open in O_RDONLY mode we get a file descriptor for the file namely fd.
Using kalloc we allocate a free memory page and read the data from the file into the memory page.
Finally, using mappages we add this newly allocated page to the page table of the process and wakeup the sleeping process.
Finally after all the process have satisfied their requests, the process will set itself up to be killed by the scheduler and voluntarily prempt itself.

Task 4: Sanity Checking

I added a testing program with the specifications given in the assignment. The name of the user program is memtest.

```c
#include "types.h"
#include "stat.h"
#include "user.h"

#define function(i) i*i*i + 3*i*i - 29*i + 5

int
main(int argc, char* argv[]){
    for(int i = 0; i < 20; i++){
        int totalBytes = 0;
        int matchedBytes = 0;
        int differentBytes = 0;

        if(fork() == 0){
            for(int j = 0; j < 10; j++){
                int *arr = malloc(4096);
                totalBytes += 4096;

                for(int k = 0; k < 1024; k++){
                    arr[k] = function(k);
                }

                for(int k = 0; k < 1024; k++){
                    if(arr[k] == (function(k)))
                        matchedBytes += 4;
                    else
                        differentBytes += 4;
                }
            }

            printf(1, "Child Number: %d\n\tTotal Bytes Allocated: %dB\n\tMatched Bytes: \
                %dB\n\tDifferentBytes: %d\n\n", i+1, totalBytes, matchedBytes, differentBytes);
            exit();
        }
        wait();
    }

    while(wait()!=-1);
    exit();
}
```

The entire specification of the user program working is given in the assignment, essentially we use fork 20 times to fork children, then run a 10 iteration loop in each child. In each iteration we dynamically allocate 4KB of memory and populate it using any random function. Then we run another iteration through the array and make sure the values were correctly stored.
These statistics are then printed by the process before exiting.
maketest.c was also added to UPROGS and EXTRA to make it a user program.

Output with PHYSTOP set to 0xE000000 (default: 240MB)

```
$ memtest
Child Number: 1
        Total Bytes Allocated: 40960B
        Matched Bytes: 40960B
        DifferentBytes: 0

Child Number: 2
        Total Bytes Allocated: 40960B
        Matched Bytes: 40960B
        DifferentBytes: 0

Child Number: 3
        Total Bytes Allocated: 40960B
        Matched Bytes: 40960B
        DifferentBytes: 0
```

For all the children, there were no differing bytes of data

Output with PHYSTOP set to 0x0400000 (4MB)

```
$ memtest
Child Number: 1
        Total Bytes Allocated: 40960B
        Matched Bytes: 40960B
        DifferentBytes: 0

Child Number: 2
        Total Bytes Allocated: 40960B
        Matched Bytes: 40960B
        DifferentBytes: 0

Child Number: 3
        Total Bytes Allocated: 40960B
        Matched Bytes: 40960B
        DifferentBytes: 0

Child Number: 4
        Total Bytes Allocated: 40960B
        Matched Bytes: 40960B
        DifferentBytes: 0
```

Still all the children had no differing bytes, indicating the implementation is correct.